

Permissioned Pools Audit



Uniswap

May 11, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	6
Security Model and Trust Assumptions	7
Privileged Roles	7
Trust Assumptions	7
Medium Severity	9
M-01 Mint Recipient Is Not Permission-Checked	9
M-02 PermissionedPositionManager Does Not Remap CONTRACT_BALANCE for Permissioned Adapter Currencies	9
M-03 Positions Can Be Opened Before Adapter Verification And Later Realize Permissioned-Token Exposure	10
M-04 User Permission Checks Can Be Bypassed When Permissioned Pools Use Non-Enforcing Hooks	11
Low Severity	11
L-01 CONTRACT_BALANCE Settlement Is Order-Dependent When Two Adapters Share One Underlying Token	11
L-02 Position Transfer Recipient Is Not Permission-Checked	12
L-03 SWEEP of Adapter Currency Can Leave Underlying Tokens in PositionManager	13
L-04 PermissionsAdapter Burns Wrapped Units Before Confirming Underlying Transfer Success	14
L-05 PermissionedPositionManager Uses the Same ERC-721 Name and Symbol as PositionManager	14
L-06 Donation Is Unavailable Through the Allowed Wrapper	15
L-07 Hook Allowlist Revocation Is Not Enforced on Liquidity Increases	16
L-08 CREATE3 Hook Address Mining Is Preemptable With a Public Deployer	16
L-09 Allowlisted Permissions Can Be Delegated Through Smart Account Execution	17
L-10 Unilateral Admin Transfer Authority in PermissionedPositionManager.transferFrom	18
Notes & Additional Information	19
N-01 Redundant Conditional in PermissionsAdapter._update	19
N-02 Typos in the Events and Errors in the IPermissionsAdapterFactory Interface	19
N-03 PermissionsAdapter Constructor Reverts For Tokens That Do Not Implement Optional ERC20 Metadata Functions	20
N-04 Permissionless Adapter Verification Does Not Prove Token Control	20
N-05 Permission Flag Evaluation Semantics Diverge Between Helpers	21
N-06 Inverted NatSpec for DeltaNotPositive/DeltaNotNegative in DeltaResolver	21
N-07 Unused ALL_ALLOWED Permission Flag in PermissionFlags	22
N-08 COMMAND_FLAG_ALLOW_REVERT Is Ineffective for COMMAND_V4_SWAP in PermissionedV4Router	23
Conclusion	24

Appendix	25
Issue Classification	25

Summary

Type	DeFi	Total Issues	22 (16 resolved)
Timeline	From 2026-03-30 To 2026-04-09	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	4 (4 resolved)
		Low Severity Issues	10 (6 resolved)
		Notes & Additional Information	8 (6 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed a review of the "Uniswap/v4-periphery" repository at the [12bd7ce](#) commit.

In scope were the following files:

```
src
├── PositionManager.sol
├── base
│   └── DeltaResolver.sol
├── hooks
│   └── permissionedPools
│       ├── PermissionedHooks.sol
│       ├── PermissionedPositionManager.sol
│       ├── PermissionedV4Router.sol
│       ├── PermissionsAdapter.sol
│       ├── PermissionsAdapterFactory.sol
│       ├── interfaces
│       │   ├── IAllowlistChecker.sol
│       │   ├── IPermissionsAdapter.sol
│       │   └── IPermissionsAdapterFactory.sol
│       └── libraries
│           └── PermissionFlags.sol
└── utils
    └── HookMinerCreate3.sol
```

System Overview

Uniswap v4 Permissioned Pools is an extension to the Uniswap v4 periphery that enables token issuers to enforce transfer restrictions on pool participants. Rather than modifying the core pool mechanics, the system introduces a wrapper token layer: each permissioned token is represented inside Uniswap v4 pools by a corresponding `PermissionsAdapter` ERC-20 token, and access control is enforced at the pool entry points through a dedicated hook contract and specialized router contracts.

The `PermissionsAdapterFactory` allows anyone to deploy a `PermissionsAdapter` for a given underlying token. Each adapter is an ERC-20 that maintains a 1:1 backing relationship with its underlying token and enforces a critical invariant: only the Uniswap v4 `PoolManager` may hold adapter token balances. The adapter achieves this by overriding its ERC-20 `_update` function so that minting is restricted to the `PoolManager` address, and any outbound transfer from the `PoolManager` automatically burns the adapter tokens and releases the equivalent underlying tokens to the recipient. A two-step verification mechanism in the factory requires the adapter to hold a non-zero balance of the underlying token before it is marked as "verified," which the hooks contract later checks when enforcing permissions.

Permission enforcement operates through two complementary layers. `PermissionedHooks` is a Uniswap v4 hook contract that intercepts `beforeSwap` and `beforeAddLiquidity` callbacks. For each pool currency that maps to a verified adapter, the hook queries an external `IAllowlistChecker` contract (configured per adapter) to verify that the originating user holds the appropriate permission flag (`SWAP_ALLOWED` or `LIQUIDITY_ALLOWED`), and also confirms that the calling wrapper contract is in the adapter's approved set.

`PermissionedV4Router` and `PermissionedPositionManager` are specialized periphery contracts that override the `_pay` function to transparently wrap underlying tokens into adapter tokens when settling with the `PoolManager`, and rely on the adapter's auto-unwrap mechanism for payouts. The `PermissionedPositionManager` additionally overrides position minting to verify that the pool's hooks are explicitly approved by each adapter owner, and overrides `transferFrom` to restrict NFT position transfers to adapter owners.

Security Model and Trust Assumptions

Privileged Roles

- **Adapter Owner** (per `PermissionsAdapter`, governed by OpenZeppelin `Ownable2Step`): Can replace the `IAllowlistChecker` contract that determines which addresses are allowed to swap or provide liquidity. Can add or remove wrapper contracts (routers, position managers) from the approved set. Can enable or disable swapping for the adapter's pools. These capabilities allow the owner to unilaterally change who may interact with the permissioned token in Uniswap v4 pools, including the ability to freeze swapping entirely or redirect allowlist evaluation to a different contract.
- **Adapter Owner as Position Transfer Authority** (in `PermissionedPositionManager`): The `transferFrom` override grants any adapter owner of either pool currency the ability to transfer any LP position NFT that contains their token, regardless of the NFT holder's consent. The function self-approves the caller and executes the transfer. This gives adapter owners custodial power over third-party liquidity positions.

Trust Assumptions

- **Allowlist checker integrity:** The external `IAllowlistChecker` contract is fully trusted to return accurate permission flags. A malicious or misconfigured checker can silently grant or deny access to any address. The only validation performed at configuration time is an ERC-165 `supportsInterface` check, which does not guarantee correct behavior.
- **Supports only standard ERC20 tokens:** The protocol assumes tokens behave like standard ERC20s (transferred amount equals balance change). Fee-on-transfer and rebasing tokens are not supported. Using them can break accounting and settlement.
- **Adapter verification as proof of token-issuer intent:** The factory's `verifyPermissionsAdapter` function treats a non-zero underlying token balance on the adapter as evidence that the adapter was authorized by the token issuer. This check can be satisfied by any party transferring tokens to the adapter, so it does not

cryptographically bind verification to token-issuer control. The system assumes only the authorized parties will transfer tokens to the adapter.

- **Wrapper contracts are trusted intermediaries:** Adapter owners explicitly approve which wrapper contracts (routers, position managers) may call `wrapToPoolManager`. Once approved, these wrappers are trusted to correctly relay the originating user's identity via `msgSender()` for allowlist checks. A wrapper that misreports the caller identity could bypass permission enforcement.
- **Hooks-adapter binding is configured, not enforced:** The `PermissionedPositionManager` maintains a per-currency, per-hook allowlist (`isAllowedHooks`) that adapter owners must configure. If an adapter owner registers an incorrect hook, positions can be minted into pools that lack permission enforcement. This configuration step is external to the adapter deployment flow and has no automated safeguard.

Medium Severity

M-01 Mint Recipient Is Not Permission-Checked

In the permissioned-pool flow, `PermissionedPositionManager` is used to mint position NFTs for pools that use permissioned assets. During minting, the contract verifies hook allowlisting via `_checkAllowedHooks` and then forwards the `owner` argument to `PositionManager._mint`, which assigns NFT ownership via `_mint(owner, tokenId)`.

The issue is that the mint recipient (`owner`) is not validated against permission flags before minting. In `PermissionedPositionManager._mint`, authorization is effectively tied to the caller and hook path, but there is no explicit check that the recipient is also allowlisted through the relevant `PermissionsAdapter`. As a result, an allowlisted user can mint a permissioned position to a non-allowlisted address.

Consider validating the recipient against the required permission flags (for each permissioned currency in the pool) before calling the base mint logic, and reverting when the recipient is not authorized. Consider also adding tests that cover third-party recipient minting to ensure the implemented behavior matches policy.

Update: Resolved in [pull request #524](#).

M-02 `PermissionedPositionManager` Does Not Remap `CONTRACT_BALANCE` for Permissioned Adapter Currencies

In permissioned pools, the pool-facing currency is the adapter token, while settlement may depend on underlying token balances held by the caller contract. In this codebase, `SETTLE` amount resolution is done through `_mapSettleAmount`, and `ActionConstants.CONTRACT_BALANCE` means “use this contract’s full balance for the specified currency” (`ActionConstants`). For reference, the router already contains a permissioned-specific override for this mapping in `PermissionedV4Router._mapSettleAmount`.

`PermissionedPositionManager` does not override this function and therefore inherits the default mapping from `DeltaResolver`, while `SETTLE` in

`PositionManager._handleAction` relies on that value directly. In permissioned flows, the contract can hold underlying tokens while adapter-token balance is zero (e.g., due to adapter transfer/unwrap behavior in `PermissionsAdapter._update`). As a result, `SETTLE(..., CONTRACT_BALANCE, false)` may resolve to `0`, causing `_settle` to return early without payment and leaving debt unsettled for the action batch.

Consider implementing the same verified-adapter remapping logic in `PermissionedPositionManager` that already exists in `PermissionedV4Router._mapSettleAmount`, so `CONTRACT_BALANCE` resolves to the underlying token balance for verified adapter currencies. Consider also adding tests that exercise permissioned `SETTLE` with `CONTRACT_BALANCE` in the position-manager path to ensure behavior is consistent with the router.

Update: Resolved in [pull request #525](#).

M-03 Positions Can Be Opened Before Adapter Verification And Later Realize Permissioned-Token Exposure

In order to enforce permissioned behavior, the pool's currency is the permissions adapter token, and permission checks are activated only after the adapter is marked as verified in the [factory](#). Before verification, both the hook check in `PermissionedHooks._isAllowed` and the hook-allowance check in `PermissionedPositionManager._checkAllowedHook` treat that adapter currency as non-permissioned.

Because of this, a user can open an out-of-range one-sided position before verification using only the non-permissioned side, then keep the position live until verification and price movement bring it into range. At that point, the position can gain exposure to the adapter side, and on removal the payout path can unwrap into the underlying token via `PermissionsAdapter`. This allows pre-verification positions to cross into post-verification permissioned behavior.

Consider not allowing any swaps, until the adapter has been verified both in router, position manager and the hook.

Update: Resolved in [pull request #527](#) and [#544](#).

M-04 User Permission Checks Can Be Bypassed When Permissioned Pools Use Non-Enforcing Hooks

In the permissioned-pool design, a permissioned token is wrapped through a `PermissionsAdapter`, and wrappers such as `PermissionedPositionManager` and `PermissionedV4Router` call `wrapToPoolManager` to settle funds into the pool manager. Hook logic in `PermissionedHooks` is expected to enforce per-user flags such as `LIQUIDITY_ALLOWED` and `SWAP_ALLOWED`, while adapter `allowedWrappers` only confirms that the caller contract is approved (`allowedWrappers[msg.sender]`).

In `PermissionedPositionManager._pay`, the `LIQUIDITY_ALLOWED` check is only applied when `payer == address(this)` (`branch`, `check`). For user-funded settlement paths (`payer == msgSender()`), there is no equivalent user permission check before `permit2.transferFrom` and `wrapToPoolManager`. If a pool is configured with a non-enforcing hook (including `address(0)` when allowed via `setAllowedHook` and checked through `isAllowedHooks[currency][hooks]`), a user without `LIQUIDITY_ALLOWED` can still provide liquidity because adapter-level checks validate the wrapper contract, not the end user. The same pattern appears in `PermissionedV4Router._pay` for `SWAP_ALLOWED` (`check branch`, `user-funded transfer/wrap`).

Consider enforcing permission-flag validation for `msgSender()` in both payer modes before any transfer/wrap path for verified adapters, so authorization does not depend on hook behavior. Consider also treating hook checks as additional protection rather than the only enforcement layer, and preventing no-hook permissioned pools unless explicitly supported and covered by tests.

Update: Resolved in [pull request #523](#).

Low Severity

L-01 CONTRACT_BALANCE Settlement Is Order-Dependent When Two Adapters Share One Underlying Token

In `PermissionedV4Router._mapSettleAmount`, when `CONTRACT_BALANCE` is used for a permissioned currency, the router resolves the amount from its balance of the underlying

token returned by `verifiedPermissionsAdapterOf`. Because `PermissionsAdapterFactory` does not enforce uniqueness of underlying token per adapter, two adapters can point to the same underlying token.

A realistic flow is a batched swap where the router accumulates credits for adapter `A` and adapter `B` (both wrapping the same underlying `U`), then executes `SETTLE(A, CONTRACT_BALANCE)` followed by `SETTLE(B, CONTRACT_BALANCE)`. The first settle uses `_pay` and `wrapToPoolManager` to consume the shared `U` balance; the second settle then contributes zero while debt for `B` can remain open. At unlock finalization, `PoolManager.unlock` reverts with `CurrencyNotSettled`, making execution outcome depend on settle ordering and causing transaction-level self-DoS.

Consider enforcing a one-to-one mapping between underlying tokens and verified adapters in `PermissionsAdapterFactory`, or disallowing `CONTRACT_BALANCE` for permissioned settles and using debt-scoped settlement (for example, `OPEN_DELTA`) so each currency is settled deterministically in batched flows.

Update: Acknowledged, not resolved. The team stated:

A reverse mapping (permissionedToken → verifiedAdapter) to enforce uniqueness introduces a griefing vector: anyone could create an adapter for an unpermissioned token with an arbitrary allowlist checker, get it verified first, and block the legitimate issuer.

L-02 Position Transfer Recipient Is Not Permission-Checked

In the permissioned-pool model, `PermissionedPositionManager` overrides `transferFrom` so that only permissioned-token admins can move position NFTs tied to pools containing wrapped permissioned assets. Because the NFT owner controls later liquidity actions, recipient validation is part of enforcing who can hold and operate these positions.

The current transfer flow validates the caller (admin) but does not validate the `to` address before calling `super.transferFrom(...)`. This means an authorized admin can transfer a permissioned position to an address that is not allowlisted, which can bypass holder-level policy expectations and create inconsistent enforcement compared with other permissioned checks.

Consider validating `to` against the relevant permission checks for each permissioned currency in the pool before completing the transfer, and reverting if the recipient is not authorized.

Consider also adding tests for admin-initiated transfers to unauthorized recipients (including mixed permissioned/non-permissioned pools) so the intended transfer policy is explicitly enforced.

Update: Resolved in [pull request #532](#). The team stated:

| *Checking the allowlist for the `to` address for both pool's tokens*

L-03 SWEEP of Adapter Currency Can Leave Underlying Tokens in PositionManager

In permissioned pools, users may interact with adapter currency, while value can be delivered as the underlying token. In [PermissionsAdapter._update](#), transfers from the PoolManager trigger adapter-side unwrapping and underlying token transfer to the recipient. This means a contract can finish a flow holding underlying tokens even if the flow is expressed in adapter currency.

In [PositionManager._handleAction](#), [SWEEP](#) forwards to [PositionManager._sweep](#), which only transfers the exact currency provided by the caller. If an integrator executes [TAKE\(adapter, ADDRESS_THIS, ...\)](#) (see [TAKE](#), [ADDRESS_THIS](#)) and then sweeps the adapter currency, the sweep can transfer zero while underlying tokens remain in the PositionManager. Because [modifyLiquidities](#) is externally callable, leftover balances may later be swept by another caller.

Consider normalizing [SWEEP](#) for verified adapters by resolving adapter input to underlying before balance checks, or reverting when [SWEEP](#) is called with adapter currency so callers must explicitly sweep the underlying token. Consider also documenting this behavior and adding tests for [TAKE\(adapter -> ADDRESS_THIS\)](#) followed by [SWEEP](#) to reduce integration mistakes.

Update: Resolved in [pull request #533](#). The team stated:

- Made [PositionManager._sweep](#) `virtual`.
- Overrode `_sweep` in [PermissionedPositionManager](#): for verified adapter currencies, sweep the underlying token instead.

L-04 PermissionsAdapter Burns Wrapped Units Before Confirming Underlying Transfer Success

`PermissionsAdapter` is the pool-facing wrapper for a permissioned underlying token. During withdrawals, the unwrap path burns wrapped units and then transfers the underlying token to the recipient via `_unwrap`. Users therefore rely on this sequence to redeem value from wrapper units.

Given the invariant that `balanceOf(P00L_MANAGER) == totalSupply` in `PermissionsAdapter`, this is not a reserve-mismatch issue. The practical failure mode is transfer-level blocking on the underlying token (for example, token pause logic), or ERC-20 implementations that return `false` instead of reverting under `IERC20.transfer`. In that case, burn can execute while payout does not, leaving the recipient without underlying tokens.

Consider using a safe transfer helper (for example, `SafeERC20.safeTransfer`) or explicitly checking the return value of `transfer` and reverting on failure, so burn and underlying payout remain atomic when transfer restrictions are active.

Update: Resolved in [pull request #534](#). The team stated:

Replaces the bare `PERMISSIONED_TOKEN.transfer` call in `PermissionsAdapter._unwrap` with `SafeERC20.safeTransfer` so burn + payout stay atomic.

L-05 PermissionedPositionManager Uses the Same ERC-721 Name and Symbol as PositionManager

The [Uniswap v4 periphery repository](#) includes a standard position manager and a permissioned variant (`PositionManager.sol`, `PermissionedPositionManager.sol`). Both contracts mint ERC-721 tokens that represent liquidity positions and are intended to be consumed by wallets, explorers, indexers, and internal reporting pipelines.

`PermissionedPositionManager` currently inherits the same ERC-721 collection metadata as `PositionManager`, so both collections present the same name and symbol.

`PositionManager` sets the metadata via `ERC721Permit_v4("Uniswap v4 Positions NFT", "UNI-V4-POSM")`, and `PermissionedPositionManager` invokes that same base constructor without overriding token metadata ([constructor call](#)). Although token identity is still unique on-chain by contract address, identical metadata can create ambiguity in off-chain

rendering and analytics systems that display or group assets by collection name/symbol, which can lead to operator and user confusion when reviewing positions, approvals, or portfolio views.

Consider assigning distinct ERC-721 metadata to `PermissionedPositionManager` (for example, a unique name/symbol suffix) and ensuring its descriptor output is clearly differentiated, so off-chain tools can reliably present permissioned positions as a separate collection.

Update: Resolved in [pull request #529](#).

L-06 Donation Is Unavailable Through the Allowed Wrapper

Permissioned-token flows for these pools are mediated by the adapter and approved wrappers. The adapter can mint wrapped balance to the pool manager only when called by an approved wrapper (`PermissionsAdapter.wrapToPoolManager`), and hook checks separately require the router/wrapper itself to be allowlisted (`PermissionedHooks._isAllowed`). This means users cannot directly wrap underlying permissioned tokens unless they are explicitly configured as wrappers.

Although `DONATE` is defined, it is marked as unsupported in router/position-manager flows (`Actions.DONATE`). In execution, `V4Router` does not implement donate and reverts unsupported actions (`V4Router._handleAction`), while `PermissionedV4Router` dispatches only Permit2 and swap commands (`PermissionedV4Router._dispatch`). As a result, no regular end user, including allowlisted users, has an exposed path to wrap and donate underlying tokens to that permissioned pool through the allowed wrapper flow.

Consider adding a dedicated donation endpoint in the permissioned router/wrapper that transfers underlying tokens to the adapter, performs wrapping through authorized wrapper logic, and forwards the donate call while preserving existing allowlist and wrapper checks.

Update: Acknowledged, not resolved. The team stated:

This is a good point. We will likely add the ability to do this in a future version

L-07 Hook Allowlist Revocation Is Not Enforced on Liquidity Increases

The permissioned-pool design uses two complementary controls: hook selection is managed in `PermissionedPositionManager` through `setAllowedHook` and the `isAllowedHooks` mapping, while runtime user and wrapper checks are enforced in `PermissionedHooks`. As a result, the position manager's hook mapping determines which hooks are permitted for position lifecycle actions.

The issue is that hook allowlist validation is applied only when minting. During mint, `_mint` calls `_checkAllowedHooks(poolKey)`, but liquidity-increase paths (`INCREASE_LIQUIDITY` and `INCREASE_LIQUIDITY_FROM_DELTAS`) use inherited flows in `PositionManager` and proceed to `_modifyLiquidity` without re-checking `isAllowedHooks`. This allows existing positions to be increased even after a hook has been removed from the allowlist, and `PermissionedHooks` does not prevent this because it does not reference `isAllowedHooks`.

Consider re-validating `_checkAllowedHooks(poolKey)` on all liquidity-increase paths for existing positions (for example, by overriding `_increase` and `_increaseFromDeltas` in `PermissionedPositionManager` before calling `_modifyLiquidity`), and documenting whether hook revocation is intended to apply only to new mints or also to future increases on existing positions.

Update: Resolved in [pull request #537](#). The team stated:

Fixed by checking `allowedHooks` on liquidity increases to prevent users from increasing their liquidity on hooks that have already been removed by the admin. Does not check on decreases to allow them to exit.

L-08 CREATE3 Hook Address Mining Is Preemptable With a Public Deployer

In Uniswap v4, hook permissions are encoded in the low bits of the hook address, so deployers often mine salts to obtain an address with the required flags.

`HookMinerCreate3::find` and `HookMinerCreate3::findWithPrefix` perform this search, and `HookMinerCreate3::computeAddress` derives the target address through `CREATE3.getDeployed`. As a result, deployment safety depends directly on control of the `(deployer, salt)` pair.

The current guidance recommends a shared public deployer (`0x4e59...`) in script contexts. If the mined salt becomes observable, another actor can deploy first with the same `(deployer, salt)`, which can make the intended deployment fail or place unintended runtime code at the mined address. In addition, availability checks in the miner rely on `code.length == 0` (`HookMinerCreate3`, `HookMinerCreate3`), which does not guarantee deployability for CREATE3 because an address may still be undeployable when its nonce is nonzero.

Consider using a deployer controlled by the hook owner instead of a shared public deployer, and keeping salts private until deployment. Consider requiring a pre-deployment check that confirms both `code.length == 0` and `nonce == 0` (for example, via `eth_getCode` and `eth_getTransactionCount`), and verifying the deployed runtime code hash before using the hook address in pool initialization.

Update: *Acknowledged, not resolved. The team stated:*

We are the ones deploying the hook, which should only be deployed once, so this shouldn't be an issue.

We will just move Create3 to a test folder.

L-09 Allowlisted Permissions Can Be Delegated Through Smart Account Execution

Permissioned access is enforced at hook level and keyed to account identity. In `PermissionedHooks._beforeSwap` and `PermissionedHooks._beforeAddLiquidity`, allowlist checks are performed through `_verifyAllowlist` and `_isAllowed`, using router-derived sender identity (`PermissionedV4Router.msgSender`).

Under account delegation models such as [EIP-7702](#), a whitelisted EOA can delegate execution to contract logic that may expose broad or unrestricted execution to third parties. Even if the account owner was originally vetted, the protocol owner cannot reliably guarantee, over time, which delegated logic is active or whether that logic permits arbitrary callers. As a result, other parties can execute swaps/liquidity actions through the whitelisted account identity, which undermines the objective of a strict whitelist.

Consider defining non-delegability as an explicit requirement for permissioned flows and enforcing it on-chain (for example, rejecting programmable/delegated accounts), or

alternatively adopting an approved-wallet policy with continuous attestation if delegated execution is intended to remain supported.

Update: Acknowledged, not resolved.

L-10 Unilateral Admin Transfer Authority in `PermissionedPositionManager.transferFrom`

In the permissioned pools design, each pool currency can have its own adapter admin, and the position manager introduces a custom transfer path for these pools. In `PermissionedPositionManager.transferFrom`, admin authority is derived per currency through `__getOwner`, rather than through the NFT holder's approval flow.

The function authorizes the caller when `msg.sender` matches either currency admin, then sets `getApproved[id] = msg.sender` before calling `super.transferFrom`. Because downstream transfer checks accept approved spenders (see `__isApprovedOrOwner` and `PositionManager.transferFrom`), a single admin from either token side can transfer a user's position NFT to an arbitrary recipient without holder consent. In addition, no dedicated admin transfer event is emitted (the contract specific event is `AllowedHooksUpdated`), so admin driven seizures are not distinguishable on-chain from normal user transfers using events alone.

Consider requiring both currency admins to authorize transfers when both pool currencies are permissioned, to avoid unilateral action over shared positions. At minimum, consider documenting this trust assumption clearly in user facing materials so liquidity providers understand the admin transfer authority before depositing. Consider also emitting a dedicated event whenever this override path is used, so admin initiated transfers are explicitly observable.

Update: Resolved in [pull request #542](#) and [#543](#).

Notes & Additional Information

N-01 Redundant Conditional in `PermissionsAdapter._update`

`PermissionsAdapter` is designed so the pool manager is the only holder of adapter tokens: minting (`from == address(0)`) can only target the pool manager, and non-manager transfers (`from != POOL_MANAGER`) are rejected. During withdrawals, the pool manager transfers adapter tokens and the contract unwraps them into the underlying permissioned token for the recipient (`_unwrap(to, amount)`, `_unwrap`).

In `PermissionsAdapter._update`, once execution passes the burn path, mint path, and the `from != POOL_MANAGER revert`, the remaining path already guarantees `from == POOL_MANAGER`. Because of that, the later condition `if (from == POOL_MANAGER) { _unwrap(to, amount); }` is always true and does not change behavior. This is a redundant branch that can make the control flow less clear.

Consider removing the redundant `if (from == POOL_MANAGER)` check and calling `_unwrap(to, amount)` directly in the reachable manager-transfer path, or restructuring the function into explicit mutually exclusive branches to make the invariant clearer and reduce unnecessary bytecode.

Update: Resolved in [pull request #530](#).

N-02 Typos in the Events and Errors in the `IPermissionsAdapterFactory` Interface

`IPermissionsAdapterFactory` exposes custom errors and an event whose names use `Pemissions` instead of `Permissions` (`PemissionsAdapterAlreadyVerified`, `PemissionsAdapterNotVerified`, and `PemissionsAdapterVerified`). This harms readability, searchability, and consistency with the rest of the codebase.

Consider renaming the errors and event to use `Permissions` consistently.

Update: Resolved in [pull request #530](#).

N-03 `PermissionsAdapter` Constructor Reverts For Tokens That Do Not Implement Optional ERC20 Metadata Functions

The `PermissionsAdapter` constructor initializes token metadata through `ERC20(_getName(permissionedToken), _getSymbol(permissionedToken))`. These helpers call token metadata methods directly in `_getName` and `_getSymbol`, including direct calls to `name()` and `symbol()`. Similarly, `decimals()` directly calls `ERC20(address(PERMISSIONED_TOKEN)).decimals()`. Because ERC20 metadata functions are not universally implemented with compatible signatures, adapters can fail to deploy for tokens that omit these methods or return incompatible types.

Consider wrapping `name()`, `symbol()`, and `decimals()` reads in low-level `staticcall` (or `try/catch`) with return-data validation and sensible fallback defaults when metadata is missing or ABI-incompatible.

Update: Resolved in [pull request #538](#).

N-04 Permissionless Adapter Verification Does Not Prove Token Control

`PermissionsAdapterFactory` marks an adapter as verified, and downstream components rely on `verifiedPermissionsAdapterOf` to determine when that adapter should be treated as the permissioned-currency entry point. This verification state therefore acts as an activation signal that changes how permissioned settlement and access flows are handled across the system.

In `verifyPermissionsAdapter`, the call is permissionless and only checks that the adapter exists, is not already verified, and has a non-zero underlying-token balance ([balance check](#)). Because any external actor can transfer a minimal amount of the underlying token to the adapter and then call verification, this process does not demonstrate ownership or operational control over adapter onboarding. As a result, third parties can trigger verification earlier than intended and move integrations into verified-mode behavior before expected setup steps are completed.

Consider restricting `verifyPermissionsAdapter` to an authorized actor (for example, adapter owner, token issuer, or governance), and requiring explicit owner intent before setting `verifiedPermissionsAdapterOf`. Consider also enforcing activation preconditions (such

as required wrapper/hook configuration) or combining setup and verification into a single atomic activation flow.

Update: Acknowledged, not resolved. The team stated:

Acknowledged as intended behavior. Allowlists are implemented differently for different permissioned tokens. Wrapped permissioned token holding a permissioned token is a universal check that the wrapped permissioned token is on the allowlist.

N-05 Permission Flag Evaluation Semantics Diverge Between Helpers

The permission model uses bitmask-based `PermissionFlag` values to represent access rights, and the codebase currently exposes two evaluation paths:

`PermissionFlags::hasFlag` and `PermissionsAdapter::isAllowed`. These functions appear similar at a glance, but they encode different matching semantics for bitmasks.

`PermissionFlags::hasFlag` treats a match as any overlapping bit (`(permissions & flag) != 0`), while `PermissionsAdapter::isAllowed` requires all requested bits to be present (`(checkAllowlist(account) & permission) == permission`). With the current single-bit constants (`SWAP_ALLOWED`, `LIQUIDITY_ALLOWED`), both checks produce the same result. If future code, docs, or integrations pass multi-bit values in one argument, the two functions can diverge: `hasFlag` may return true for partial overlap, while `isAllowed` would still require full inclusion.

Consider aligning `hasFlag` with the same equality rule used by `isAllowed` when multi-bit arguments are expected, or documenting explicitly that `hasFlag` is an any-bit check and reserving `isAllowed` for all-bits authorization.

Update: Resolved in [pull request #535](#). The team stated:

Removed unused `hasFlag()` function

N-06 Inverted NatSpec for `DeltaNotPositive/DeltaNotNegative` in `DeltaResolver`

`DeltaResolver` maps signed `currencyDelta` values from `PoolManager` into actions: a negative delta means the contract owes funds and must settle, while a positive delta means

the contract is owed funds and can take. This sign-based flow is implemented in `_getFullDebt`, `_getFullCredit`, and used by `_mapSettleAmount` and `_mapTakeAmount`.

The NatSpec notices for the custom errors are inverted relative to runtime behavior. `DeltaNotPositive` is documented as “trying to settle a positive delta,” but it is raised in `_getFullCredit` when the delta is negative. Conversely, `DeltaNotNegative` is documented as “trying to take a negative delta,” but it is raised in `_getFullDebt` when the delta is positive. This documentation mismatch can mislead integrators and off-chain tooling that interpret selector intent from NatSpec.

Consider updating the NatSpec `@notice` text so `DeltaNotNegative` clearly refers to the settle path receiving a positive delta, and `DeltaNotPositive` clearly refers to the take path receiving a negative delta. Consider keeping existing selectors and revert sites unchanged for compatibility, and adding short clarifying comments near `_getFullDebt`, `_getFullCredit`, `_mapSettleAmount`, and `_mapTakeAmount`.

Update: Resolved in [pull request #530](#).

N-07 Unused `ALL_ALLOWED` Permission Flag in `PermissionFlags`

In the permissioned-pools architecture, allowlist permissions are encoded as bit flags in `PermissionFlags` and enforced at runtime through `PermissionsAdapter.isAllowed`, with callers passing operation-specific flags from components such as `PermissionedHooks`, `PermissionedV4Router`, and `PermissionedPositionManager`.

`PermissionFlags` declares `ALL_ALLOWED (0xFFFF)`, but production permission checks use only `SWAP_ALLOWED` and `LIQUIDITY_ALLOWED`. Usage of `ALL_ALLOWED` appears in test and mock code (for example, `PermissionedV4Router.t.sol` and `PermissionedRoutingTestHelpers.sol`). This leaves `ALL_ALLOWED` unused in deployed logic and may suggest a runtime permission path that is not actually exercised.

Consider removing `ALL_ALLOWED` from `PermissionFlags` and using explicit combinations (for example `SWAP_ALLOWED | LIQUIDITY_ALLOWED`) in tests to keep test setup aligned with production permission checks.

Update: Acknowledged, not resolved. The team stated:

Thanks for pointing that out! However, we have decided that we shall keep it for future use.

N-08 `COMMAND_FLAG_ALLOW_REVERT` Is Ineffective for `COMMAND_V4_SWAP` in `PermissionedV4Router`

`PermissionedV4Router.execute` is structured to process commands in a loop and decide whether a failed command should revert by checking `_successRequired(command)`. This design assumes `_dispatch` reports failures as `(success = false, output = reason)` instead of reverting, so commands marked with `COMMAND_FLAG_ALLOW_REVERT` can fail without aborting the full `execute` flow.

That behavior is preserved for `COMMAND_PERMIT2_PERMIT`, which uses a low-level `.call` and captures revert data, but not for `COMMAND_V4_SWAP`. The swap branch calls `_executeActions(inputs)`, and `_executeActions` directly calls `poolManager.unlock(unlockData)` without `try/catch` or low-level handling. As a result, swap failures (including slippage checks such as `V4TooLittleReceived` and `V4TooMuchRequested`) bubble up and revert `execute` immediately, making `COMMAND_FLAG_ALLOW_REVERT` ineffective for swap commands.

Consider modifying the swap path so it captures and returns revert data rather than bubbling reverts; for example, route swap execution through `try/catch` around `poolManager.unlock(...)` or use `address(poolManager).call(abi.encodeWithSelector(IPoolManager.unlock.selector, inputs))` and return the resulting `(success, output)` from `_dispatch`. Consider adding tests to assert that `COMMAND_V4_SWAP | COMMAND_FLAG_ALLOW_REVERT` does not revert `execute` on swap failure and that revert bytes are surfaced through `output`.

Update: Resolved in [pull request #540](#). The team stated:

Added natspec documentation to clarify behavior.

Conclusion

OpenZeppelin reviewed the Uniswap v4 Permissioned Pools extension, a set of contracts that layer token-issuer-controlled access restrictions onto standard Uniswap v4 pool operations through a wrapper token and hook-based enforcement model.

The system's architecture cleanly separates concerns: the `PermissionsAdapter` manages token wrapping and invariant enforcement, `PermissionedHooks` gates pool entry points, and the specialized router and position manager contracts handle transparent wrapping during settlement. However, the review identified several paths where permission checks are incomplete or can be circumvented. The most significant finding involves pools configured with hooks that do not enforce allowlist checks, which allows unrestricted access to permissioned tokens. Additional gaps were found around unchecked mint recipients, positions opened before adapter verification, and inconsistent enforcement during liquidity increases and position transfers. These findings indicate that the permission boundary, while architecturally coherent, does not yet cover all interaction paths through the Uniswap v4 periphery. Particular attention should be given to ensuring that every entry point that moves permissioned tokens into or out of the `PoolManager` consistently validates the end user's permissions, and that configuration-dependent safeguards (such as the hook allowlist) fail closed rather than open.

Thanks are extended to the Uniswap team for their responsiveness throughout the audit.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.